Making AI Answers Cheaper

The Energy-Efficiency-DL-Model project, explained simply

In one sentence: we make a language model (like a small ChatGPT) write its answers using **less electricity and less time** on a laptop graphics card, while giving **exactly the same answer** as before.

1. The problem

A language model writes one small piece of text at a time. Each piece is called a **token** (roughly a word or part of a word).

- To write every single token, the model runs its **whole brain**: all 16 of its processing **layers**.
- That costs energy and time **for every token**, even for easy ones like "the" or "of".
- On a small laptop GPU (an NVIDIA **RTX 3050**), this makes answers slow and power-hungry.

2. Our idea, with a simple example

Think of a **junior writer** and a **senior editor**:

- The **junior writer** (only the first few layers of the model) quickly guesses the next few words. This is cheap.
- The **senior editor** (the full model) checks all those guesses **at once**, in one go.
- Correct guesses are kept. At the first wrong guess, the editor writes the right word and the junior starts again from there.

Checking several words at once is much cheaper than writing them one by one. The editor always has the final say, so **the answer never changes**.

This technique is called **self-speculative decoding**. We use a model from Meta, **LayerSkip Llama 3.2 1B**, which was trained so that its first layers can make good guesses.

3. How it works, step by step

Step	What happens	Everyday comparison
1. Read	The model reads the question once, using all layers.	The editor reads the brief.
2. Guess	The first layers guess the next 1 to 4 words.	The junior writes a quick draft.
3. Check	The full model checks all guessed words in one pass.	The editor reviews the draft.
4. Keep	Correct words are kept; the first wrong one is corrected.	Good lines stay, the first mistake is fixed.
5. Repeat	Go back to step 2 until the answer is finished.	Next paragraph.

4. The smart part: choosing settings automatically

How many layers should guess, and how many words should they guess? There is no single best answer. It depends on the question. So the project includes a **selector** that learns:

- It **tries** each setting, **measures** the real energy, and then prefers the cheapest one.
- **Safety rule:** if any setting ever changes an answer, it is removed from the choices.
- **Adaptive guessing** (newer option): if the guesses keep being wrong (fewer than 3 in 10 correct), it stops guessing and writes normally, then tries again a little later.
- **Per question type** (newer option): maths questions and coding questions can get different settings.

The two newer options are switched off by default and were **not** part of the results on the next page.

5. How we make sure the test is fair

- **Same answer check:** every answer is compared word by word with the normal (non-guessing) answer. Any difference is recorded as a failure.
- **Real energy:** energy comes from the GPU's own built-in meter (NVIDIA NVML). It is measured, never estimated.
- **Fair order:** all methods run on the same questions, in random order, so heat or timing does not favour one method.
- **Public questions:** maths problems (GSM8K) and coding problems (HumanEval), fixed to an exact version so anyone can repeat the test.

6. Results so far

Test: RTX 3050 laptop GPU, full precision (FP32), 8 test questions, each run twice, up to 16 words per answer. **133 out of 133** answers were exactly the same as normal writing.

Method	Time per word	Energy per word	In simple words
Normal (no guessing)	112 ms	4.39 J	The starting point
Fixed guessing	86 ms	3.92 J	Always the same setting
Bandit (simple learner)	84 ms	3.64 J	Learns by trial
Adaptive (our method)	**76 ms**	**3.48 J**	**Fastest and lowest energy**

The maths (adaptive compared with normal):

- Energy: 3.48 / 4.39 = 0.79, so **about 21% less energy** per word.
- Time: 76 / 112 = 0.67, so **about 33% less time** per word.
- Speed: 112 / 76 = 1.49, so **about 1.5 times as many words per second**.

What this means: on a cheap laptop GPU, the same answers can be produced about one-fifth cheaper in energy and about one-third faster. This is an early test with a small sample, so it still needs to be repeated with more questions and longer answers.

7. One open problem

In **half precision (FP16)**, numbers are stored with fewer digits to save energy. When two possible words are almost tied, tiny rounding differences can make the checker pick the other one. One larger FP16 test stopped because of this. The code now records such cases instead of stopping, and a diagnosis tool checks whether they are only near-ties.

8. What is inside the project

The project has three parts. Only the first one is the main research.

Folder	What it is	Think of it as
Layerskip_energy	The real experiment: the guessing engine, the selector, the energy meter and the reports.	The main lab
Controller_simulation	A pretend version with made-up numbers, used to design the selector before real tests.	The practice sketch
Distillation	Teaches a small model (Qwen3 0.6B) to copy a bigger one (Qwen3 1.7B). This is called distillation.	A side project

9. The main files and their jobs

File	Its job in one line
Decoding_engine.py	Writes answers: normal, guessing, or many questions at once.
llama_model_utils.py	Runs only part of the model (first layers or last layers). Borrowed from Meta's LayerSkip.
Policy_selector.py	Decides which guessing setting to use for each question.
Energy_meter.py	Reads the real time and the GPU energy meter.
Run_experiment.py	Runs the full experiment and writes the results report.
Fixed_setting_sweep.py	Tests every fixed setting one by one, for comparison.
Observer_feedback.py	Checks which extra observations from inside the model are valid.
Cuda_graph_drafter.py	Experimental speed-up for the guessing step on NVIDIA GPUs.
Gpu_pinning.py	Holds the GPU speed steady so energy numbers can be repeated.
Diagnose_mismatch.py	Explains why an answer changed (near-tie or real bug).
Tests/	57 automatic checks that the code still works correctly.
Results/	Saved reports from every experiment, grouped by test.

10. How to run it

From the project folder (full commands are in **HOW_TO_RUN.txt**):

Goal	Command
Install (Windows + RTX 3050)	powershell -ExecutionPolicy Bypass -File .\Scripts\Install-CudaEnvironment.ps1
Check everything works	python -m pytest -q Layerskip_energy Controller_simulation
See a quick demo	python -m Controller_simulation.Demo
Run the real experiment	python -m Layerskip_energy.Run_experiment --device cuda

11. What comes next

- Repeat the test with **more questions and longer answers** to make the result stronger.
- Find out whether the FP16 differences are only **near-ties**.
- Try the newer speed-ups on the RTX 3050: **adaptive guessing**, **batching** (several questions at once) and **CUDA Graphs**.
- Combine with the **smaller distilled model** for even lower energy.

12. Small glossary

Word	Meaning
Token	A small piece of text, about one word.
Layer	One processing step inside the model. This model has 16.
GPU	The graphics card that runs the model.
Joule (J)	A unit of energy. Lower energy per word is better.
Latency	How long it takes to produce one word.
Greedy decoding	Always picking the single most likely next word. Our "normal" baseline.
Speculative decoding	Guess several words cheaply, then check them all at once.
FP32 / FP16	Full and half precision: how many digits each number keeps.
Distillation	Training a small model to imitate a bigger one.